



UNIDADE III

Programação
Estruturada em C

Prof. Me. Fábio Assis

Vetores em C

- Vetores (ou *arrays* unidimensionais) são estruturas que armazenam vários valores do mesmo tipo em posições sequenciais (ou consecutivas) da memória.
- Cada elemento do vetor é acessado por meio de um índice numérico, que em C sempre começa em 0 e vai até tamanho $- 1$.
- Exemplo: Imagine um conjunto de caixas numeradas (índice) de 0 até $n - 1$, sendo $n = 6$ a quantidade total de caixas. Cada caixa guarda um número (valor).



Vetores em C

- Declaração do Vetor: Ao declararmos um vetor na linguagem de programação C, devemos obrigatoriamente dizer qual é o tipo a ele associado.
- Vetores podem ser de qualquer tipo de dado válido, ou seja, qualquer tipo de dado que possa ser usado para declarar variáveis também pode ser usado para declarar vetores.

Definição básica: Em C um vetor deve ter: *tipo* *nome_do_vetor* [*tamanho*];

(identificador)

`int notas[5];`

Podemos acessar suas posições da seguinte forma (nome e posição):

`notas[0], notas[1], notas[2], notas[3], notas[4]`

- O índice sempre começa em 0, portanto, em um vetor de tamanho 5, o último elemento estará na posição 4 (ou seja, tamanho - 1).

Vetores em C

- Preenchimento dos Valores do Vetor: Podemos preencher os valores do vetor tanto na declaração do vetor quanto posteriormente.

Preenchendo o vetor (inicializando na declaração):

```
int idades[3] = {10, 20, 30};
```

- Esse vetor foi declarado do tipo inteiro (int), com o nome de idades e inicializado com os valores 10, 20 e 30.

Preenchendo o vetor (inicializando após a declaração):

```
int notas[2];  
notas[0] = 10; // primeiro elemento  
notas[1] = 30; // último elemento
```

- Cuidado: Em C, não há verificação automática de limites, ou seja, não há proteção contra acesso fora dos limites do vetor.

Vetores em C

- Tamanho do vetor definido pelo usuário: Podemos solicitar ao usuário para digitar o tamanho do vetor e, depois de termos esse valor, declaramos o vetor e passamos à variável que possui o tamanho ao vetor.

Exemplo:

```
int tam;
```

```
printf("Digite o tamanho do vetor: ");  
scanf("%d", &tam); //tam armazena o tamanho do vetor
```

```
int vetor[tam];
```

- Se o usuário digitasse 3, seria armazenado este valor na variável *tam*, conseqüentemente o vetor, ao ser criado, teria o tamanho 3.

Vetores em C

- Preenchendo o vetor **sem** laço de repetição: É possível preencher os valores de um vetor um a um, sem usar laços de repetição. No entanto, esse método torna o código repetitivo, extenso e menos eficiente, não sendo recomendado para vetores grandes.

Exemplo: Leitura múltipla com um único scanf, mas ainda sem uso de laço de repetição:

```
int i, nota[3];

printf("Digite 3 notas: ");
scanf("%d %d %d", &nota[0], &nota[1], &nota[2]);

//Ou com vários comandos um por linha
printf("Digite a primeira nota: ");
scanf("%d", &nota[0]);

printf("Digite a segunda nota: ");
scanf("%d", &nota[1]);

printf("Digite a terceira nota: ");
scanf("%d", &nota[2]);
```

Interatividade

Analise o código abaixo escrito em C e as afirmações a seguir:

```
#include <stdio.h>

int main() {
    int i, nota[4];
    int soma = 0;
    float media;

    for(i = 0; i < 4; i++) {
        printf("Digite a %dª nota: ", i + 1);
        scanf("%d", &nota[i]);
        soma += nota[i];
    }

    media = soma / 4.0;

    printf("A média do aluno é: %.2f\n", media);

    return 0;
}
```

Interatividade

Após análise do código escrito em C, leia as afirmações abaixo:

- I. O programa utiliza um vetor de inteiros para armazenar as quatro notas.
- II. A variável soma é acumulada dentro do laço de repetição.
- III. O uso de 4.0 na divisão garante que a média seja do tipo float.

Está correto o que se afirma em:

- a) I e II, apenas.
- b) I e III, apenas.
- c) I, apenas.
- d) I, II e III.
- e) Nenhuma das alternativas anteriores.

Resposta

Após análise do código escrito em C, leia as afirmações abaixo:

- I. O programa utiliza um vetor de inteiros para armazenar as quatro notas.
- II. A variável soma é acumulada dentro do laço de repetição.
- III. O uso de 4.0 na divisão garante que a média seja do tipo float.

Está correto o que se afirma em:

- a) I e II, apenas.
- b) I e III, apenas.
- c) I, apenas.
- d) I, II e III.
- e) Nenhuma das alternativas anteriores.

Vetores em C

- Preenchimento de vetores **com** laço de repetição: Prefira utilizar estruturas de repetição para preencher os valores de um vetor, evitando repetições desnecessárias de código.

Exemplo:

```
#include <stdio.h>

int main() {
    int i, nota[3];

    for(i = 0; i < 3; i++) {
        printf("Digite a %dª nota: ", i + 1);
        scanf("%d", &nota[i]);
    }

    return 0;
}
```

Vetores em C

- Impressão de vetor: Imprimir um vetor na linguagem C significa mostrar na tela os valores armazenados em suas posições de memória. Como um vetor contém vários elementos, é necessário acessar cada posição individualmente, informando cada uma delas ou utilizando um laço de repetição.

Exemplo sem uso de laço de repetição:

- Durante esse processo, usamos o índice do vetor para indicar qual elemento será exibido, e a função printf() para realizar a impressão no console (saída na tela).

```
#include <stdio.h>
```

```
int main() {  
    int nota[2];
```

```
    nota[0] = 5;
```

```
    nota[1] = 8;
```

```
    printf("Nota 1: %d\n", nota[0]);
```

```
    printf("Nota 2: %d\n", nota[1]);
```

```
    return 0;
```

```
}
```

Vetores em C

- O uso de laço `for` é o método mais comum para percorrer o vetor, permitindo acessar e imprimir cada item com precisão, independentemente do tamanho do vetor.
- Exemplo **com** uso de laço de repetição: Impressão com laço de repetição `for`.
- As variáveis *i*, *tam* e *nota*, devem ser declaradas antes do *for*.

```
#include <stdio.h>

int main() {
    int i, tam=5;
    int nota[tam];

    for(i=0; i<tam; i++) {
        nota[i]=6+i;
    }
    for(i=0; i<tam; i++) {
        printf("Nota %d: %d\n", i+1, nota[i]);
    }

    return 0;
}
```

Interatividade

Analise o código abaixo escrito em C. Marque a alternativa que apresenta a saída na tela deste programa.

- a) 0 1 2 3 4.
- b) 1 2 3 4 5.
- c) 2 4 6 8 10.
- d) 5 4 3 2 1.
- e) 1 3 5 7 9.

```
#include <stdio.h>

int main() {
    int i, tam=5;
    int nota[tam];

    for (i=0; i<tam; i++) {
        nota[i]=i+1;
    }
    for (i=tam-1; i>=0; i--) {
        printf("%d ", nota[i]);
    }

    return 0;
}
```

Resposta

Analise o código abaixo escrito em C. Marque a alternativa que apresenta a saída na tela deste programa.

- a) 0 1 2 3 4.
- b) 1 2 3 4 5.
- c) 2 4 6 8 10.
- d) 5 4 3 2 1.**
- e) 1 3 5 7 9.

```
#include <stdio.h>
```

```
int main() {  
    int i, tam=5;  
    int nota[tam];  
  
    for (i=0; i<tam; i++) {  
        nota[i]=i+1;  
    }  
    for (i=tam-1; i>=0; i--) {  
        printf("%d ", nota[i]);  
    }  
  
    return 0;  
}
```

Matriz em C

- Matrizes são vetores com duas dimensões: linhas e colunas. Elas são usadas para representar tabelas, grades, mapas, ou qualquer estrutura que envolva dois eixos (linha, coluna), como: mapas e tabuleiros.
- Matriz é um tabuleiro com linhas e colunas. Ideal para mapas, fases, batalhas.

Sintaxe da declaração de matriz: *tipo nomeMatriz[tamanhoLinha][tamanhoColuna]*:

Declarando uma matriz:

- `int matriz1[4][3];`

		Colunas		
Linhas	índice	0	1	2
	0			
	1			
	2			
	3			

Matriz em C

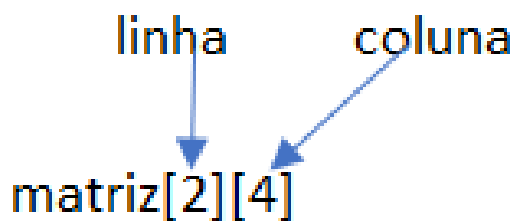
- Em uma matriz do tipo `matriz[lin][col]`, o primeiro índice representa a linha e o segundo índice representa a coluna.

`matriz[2][2]`

	<code>coluna0</code>	<code>coluna1</code>
<code>linha0</code>	<code>matriz[0][0]</code>	<code>matriz[0][1]</code>
<code>linha1</code>	<code>matriz[1][0]</code>	<code>matriz[1][1]</code>

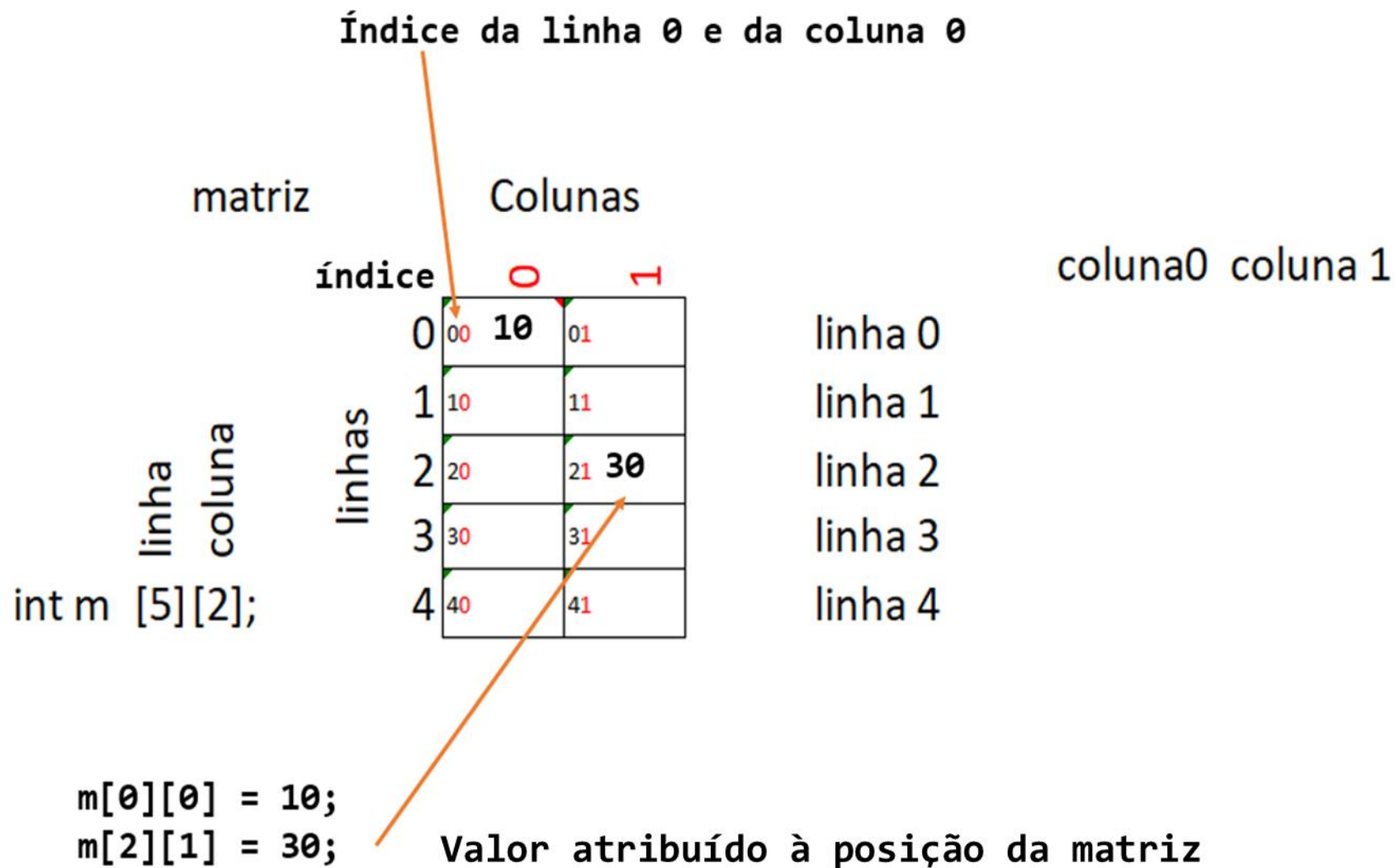
`matriz[2][4]`

	<code>coluna0</code>	<code>coluna1</code>	<code>coluna2</code>	<code>coluna3</code>
<code>linha0</code>	<code>matriz[0][0]</code>	<code>matriz[0][1]</code>	<code>matriz[0][2]</code>	<code>matriz[0][3]</code>
<code>linha1</code>	<code>matriz[1][0]</code>	<code>matriz[1][1]</code>	<code>matriz[1][2]</code>	<code>matriz[1][3]</code>



Matriz em C

- Acessando os elementos da matriz: Primeiro acessamos a linha, depois a coluna.



Matriz em C

Preenchendo os valores da matriz:

Inicialização direta com valores:

Você pode preencher a matriz no momento da declaração:

```
int m[2][2] = {{7, 4}, {2, 9}};
```

Ou, ainda, sem os colchetes internos explícitos (forma equivalente):

```
int m[2][2] = {7, 4, 2, 9};
```

Preenchimento manual por posição (exemplo com scanf).

Permite que o usuário digite os valores:

```
scanf("%d", &m[0][0]);
```

Matriz em C

Preenchendo os valores com laço de repetição: Mais prático e escalável, recomendado para matrizes maiores:

```
int i, j;
for(i = 0; i < 2; i++) {
    for(j = 0; j < 2; j++) {
        printf("Digite o valor da posição [%d][%d]: ", i, j);
        scanf("%d", &m[i][j]);
    }
}
```

Interatividade

Analise o código abaixo escrito em C e as afirmações:

- I. A matriz m possui 2 linhas e 3 colunas, totalizando 6 elementos.
- II. O laço externo com a variável i percorre as linhas da matriz.
- III. O valor armazenado na posição m[1][2] é o número 2.

Está correto o que se afirma em:

- a) I e II, apenas.
- b) I e III, apenas.
- c) I, apenas.
- d) I, II e III.
- e) II, apenas.

```
#include <stdio.h>

int main() {
    int m[2][3] = {{3, 4, 5}, {1, 6, 2}};
    int i, j;

    for(i = 0; i < 2; i++) {
        for(j = 0; j < 3; j++) {
            printf("%d ", m[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Resposta

Analise o código abaixo escrito em C e as afirmações:

- I. A matriz m possui 2 linhas e 3 colunas, totalizando 6 elementos.
- II. O laço externo com a variável i percorre as linhas da matriz.
- III. O valor armazenado na posição m[1][2] é o número 2.

Está correto o que se afirma em:

- a) I e II, apenas.
- b) I e III, apenas.
- c) I, apenas.
- d) I, II e III.
- e) II, apenas.

```
#include <stdio.h>

int main() {
    int m[2][3] = {{3, 4, 5}, {1, 6, 2}};
    int i, j;

    for(i = 0; i < 2; i++) {
        for(j = 0; j < 3; j++) {
            printf("%d ", m[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Matriz em C: exemplos

- Manipulando a matriz com uso de laço de repetição:
- Exemplo: Este programa cria uma matriz com 3 linhas e 3 colunas.
- Em seguida, solicita ao usuário que digite um valor para cada posição da matriz. Por fim, imprime todos os valores da matriz em formato de tabela.

```
int i, j;
int lin = 3, col = 3;
int matriz[lin][col];

// Leitura dos valores informados pelo usuário
for(i = 0; i < lin; i++)
    for(j = 0; j < col; j++) {
        printf("Digite o valor da posição [%d][%d]: ", i, j);
        scanf("%d", &matriz[i][j]);
    }

// Impressão da matriz
printf("\nMatriz digitada:\n");
for(i = 0; i < lin; i++) {
    for(j = 0; j < col; j++){
        printf("%d ", matriz[i][j]);
    }
    printf("\n");
}
```

Matriz em C: exemplos

- Exemplo: Programa para somar todos os elementos de uma matriz com valores predefinidos e mostrar o resultado na tela.
- O programa percorre cada valor e acumula na variável soma.

Saída na tela:

Soma total = 24

```
#include <stdio.h>

int main() {
    int i, j, soma=0;

    int m[3][4]={{1, 2, 3, 4},
                 {1, 1, 1, 1},
                 {4, 3, 2, 1}};

    for(i=0; i<3; i++){
        for(j=0; j<4; j++){
            soma += m[i][j];
        }
    }
    printf("Soma total = %d\n", soma);

    return 0;
}
```

Matriz em C: exemplos

- Exemplo: Este programa armazena as notas de 4 alunos, sendo que cada aluno possui 3 notas. Em seguida, calcula a média individual de cada aluno e exibe o resultado formatado na tela.

```
float notas[4][3], soma, media;
int i, j;

// Leitura das notas
for(i = 0; i < 4; i++) {
    printf("Aluno %d:\n", i + 1);
    for(j = 0; j < 3; j++) {
        printf("Digite a %dª nota: ", j + 1);
        scanf("%f", &notas[i][j]);
    }
}

// Cálculo e exibição da média de cada aluno
printf("\nMédias dos alunos:\n");
for(i = 0; i < 4; i++) {
    soma = 0;
    for(j = 0; j < 3; j++) {
        soma += notas[i][j];
    }
    media = soma / 3.0;
    printf("Aluno %d: Média = %.2f\n", i + 1, media);
}
```


Interatividade

Analise o código abaixo escrito em C e as afirmações:

- I. A matriz m possui 2 linhas e 2 colunas, totalizando 4 elementos.
- II. A variável maior foi iniciada com o valor 0.
- III. O maior valor é o número 7 e está na posição m[0][1].

```
#include <stdio.h>
```

Está correto o que se afirma em:

- a) I e II, apenas.
- b) I e III, apenas.
- c) I, apenas.
- d) I, II e III.
- e) II, apenas.

```
int main() {  
    int m[2][2] = {{4, 7}, {1, 5}};  
    int maior = m[0][0];  
  
    for (int i = 0; i < 2; i++)  
        for (int j = 0; j < 2; j++)  
            if (m[i][j] > maior)  
                maior = m[i][j];  
  
    printf("Maior valor: %d\n", maior);  
    return 0;  
}
```

Resposta

Analise o código abaixo escrito em C e as afirmações:

- I. A matriz m possui 2 linhas e 2 colunas, totalizando 4 elementos.
- II. A variável maior foi iniciada com o valor 0.
- III. O maior valor é o número 7 e está na posição m[0][1].

```
#include <stdio.h>
```

Está correto o que se afirma em:

- a) I e II, apenas.
- b) I e III, apenas.
- c) I, apenas.
- d) I, II e III.
- e) II, apenas.

```
int main() {  
    int m[2][2] = {{4, 7}, {1, 5}};  
    int maior = m[0][0];  
  
    for (int i = 0; i < 2; i++)  
        for (int j = 0; j < 2; j++)  
            if (m[i][j] > maior)  
                maior = m[i][j];  
  
    printf("Maior valor: %d\n", maior);  
    return 0;  
}
```

Referências

- BACKES, André. *Linguagem C: completa e descomplicada*. São Paulo: Grupo GEN, 2018.
- CELES, Waldemar; RANGEL, Renato; EIRAS, José. *Introdução a estruturas de dados: com técnicas de programação em C*. Rio de Janeiro: Elsevier/Grupo GEN, 2016.
- DAMAS, Luiz. *Linguagem C*. 10. ed. Rio de Janeiro: LTC, 2011.
- DEITEL, Paul; DEITEL, Harvey. *C: como programar*. 7. ed. São Paulo: Pearson Prentice Hall, 2013.
- MANZANO, José Augusto N. G.; OLIVEIRA, Jayr Figueiredo de. *Algoritmos: lógica para desenvolvimento de programação de computadores*. São Paulo: Editora Saraiva, 2016.

ATÉ A PRÓXIMA!